

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

COMBINATORIAL OPTIMIZATION

Pham Quang Dung

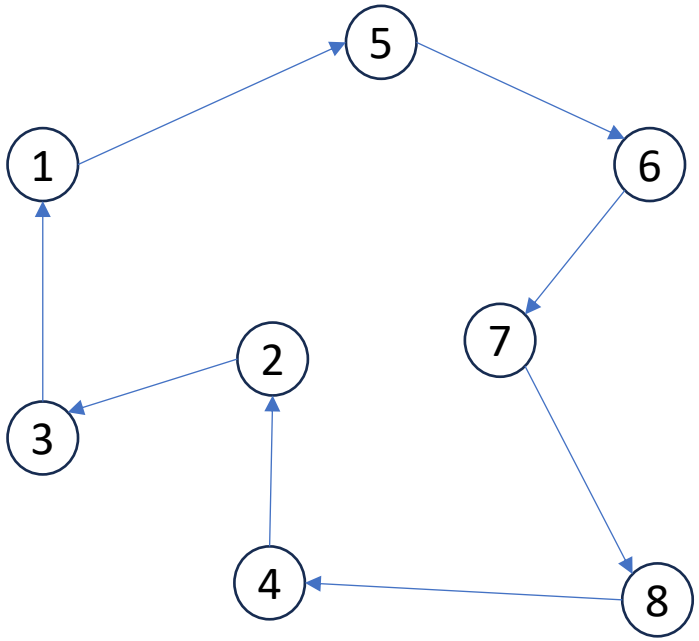
ONE LOVE. ONE FUTURE.

CONTENT

- Introduction to combinatorial optimization & applications
- The TSP problem (description & modelling)
- Exact methods
  - Exhaustive search with backtracking
  - Branch and Bound
  - Constraint Programming (with OR-tools)
  - Integer Linear Programming (with OR-tools)
- Incomplete methods
  - Swap-based neighborhood: hill climbing and tabu search
  - 2-opt neighborhood: hill climbing and tabu search

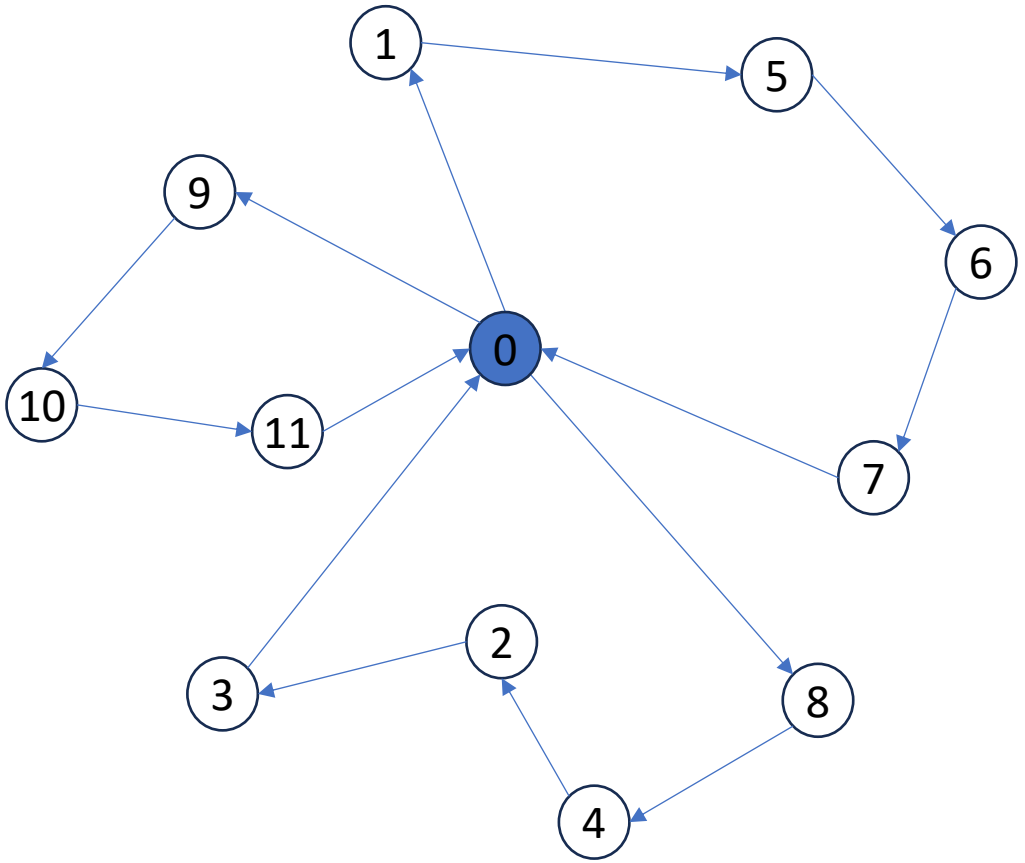
Introduction to combinatorial optimization & applications

- Travelling Salesman Problem (TSP) Find the shortest closed tour visiting all the given points



Introduction to combinatorial optimization & applications

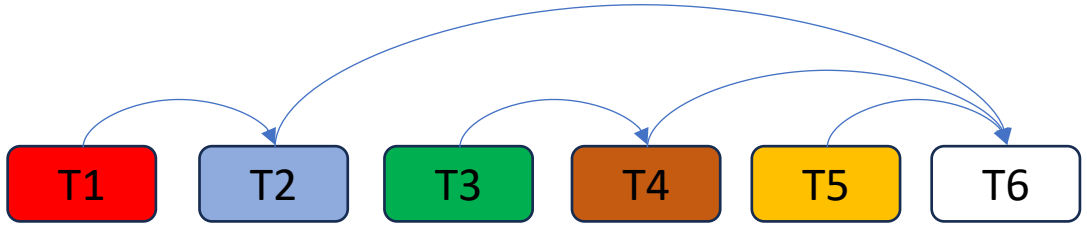
- Capacitated Vehicle Routing Problem (CVRP) Find routes for *k* capacitated vehicles to deliver packages from a central depot

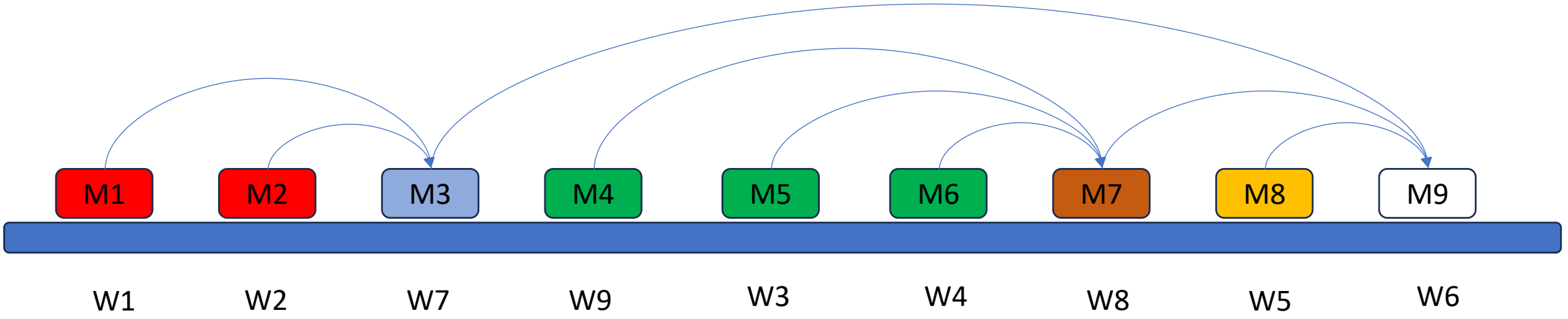


- Line Scheduling

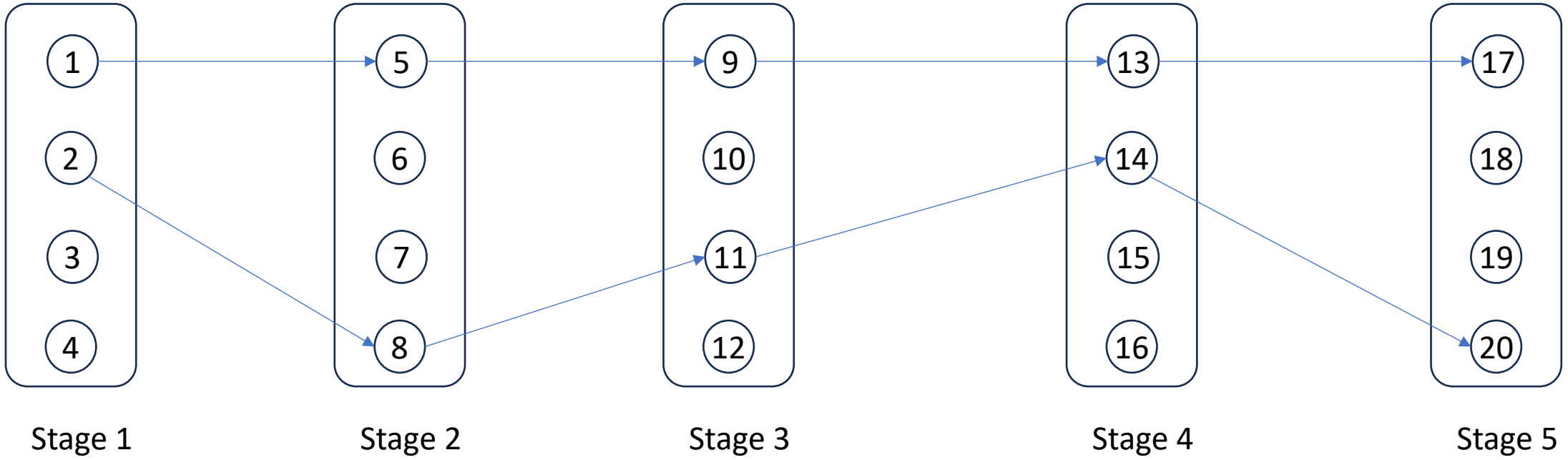
	1	2	3	4	5	6	7	8	9
1	4	5						4	
2		6		5		2	4		
3			5	7					6
4		8				5		2	
5			9		6			5	
6			4			9	2		

	1	2	3	4	5	6	7	8	9
1	1	1							
2			1						
3				1	1	1			
4							1		
5								1	
6									1





- Solution construction
  - Sequence of stages, at each stage → select a choice among possible options



- Linear Integer Programming

Linear program

$$f(x) = c_1x_1 + c_2x_2 + \ldots + c_nx_n \rightarrow \max$$

subject to

$$a_{1,1}x_1 + a_{1,2}x_2 + \ldots + a_{1,n}x_n \leq b_1$$
$$a_{2,1}x_1 + a_{2,2}x_2 + \ldots + a_{2,n}x_n \leq b_2$$
$$\ldots$$
$$a_{m,1}x_1 + a_{m,2}x_2 + \ldots + a_{m,n}x_n \leq b_m$$
$$x_1, x_2, \ldots x_n \in R, x_1, x_2, \ldots x_n \geq 0$$

Linear integer program

$$f(x) = c_1x_1 + c_2x_2 + \ldots + c_nx_n \rightarrow \max$$

subject to

$$a_{1,1}x_1 + a_{1,2}x_2 + \ldots + a_{1,n}x_n \leq b_1$$
$$a_{2,1}x_1 + a_{2,2}x_2 + \ldots + a_{2,n}x_n \leq b_2$$
$$\ldots$$
$$a_{m,1}x_1 + a_{m,2}x_2 + \ldots + a_{m,n}x_n \leq b_m$$
$$x_1, x_2, \ldots x_n \in Z, x_1, x_2, \ldots x_n \geq 0$$

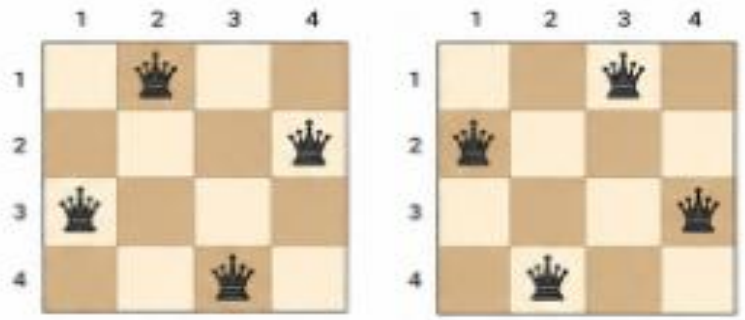
- **Problem description**
  - A company must decide to make a plan to produce 2 products P_1, P_2.
    - The revenue received when selling 1 unit of P_1 and P_2 are respectively 5$ and 7$
    - The manufacturing cost when producing P_1 and P_2 are respectively 5$ and 3$
    - The storage cost in warehouses for 1 unit of P_1 and P_2 are respectively 2$ and 3$
  - Compute the production plan so that
    - Total manufacturing cost is less than or equal to 200$
    - Total storage cost is less than or equal to 150$
    - Total revenue is maximal
- **Linear Programming Model**
  - Decision variables: X_1 and X_2: amount of products P_1 and P_2 to be produced
  - Constraints:
    - Manufacturing cost: $5X_1 + 3X_2 \leq 200$
    - Storage cost: $2X_1 + 3X_2 \leq 150$
  - Objective: maximize $5X_1 + 7X_2$

Introduction to combinatorial optimization & applications

- Combinatorial Optimization Problem (COP): min (max) $\{f(x): x \in P\}$
- (COP) = $\{X, D, C, f\}$
  - $X = \{X_1, X_2, \ldots, X_n\}$ - decision variables
  - $D = \{D(X_1), D(X_2), \ldots, D(X_n)\}$ - domains of variables
  - $C = \{C_1, C_2, \ldots, C_k\}$ – set of constraints
  - $f(X)$: objective function to be optimized

Introduction to combinatorial optimization & applications

- **n-queens** how to place n queens in a chess board such that no two queens attack each other
  - Decision variables
  - $X = (X_1, X_2, \ldots, X_n)$
  - Domain: $D(X_i) = \{1, 2, \ldots, n\}$
  - Constraints
    - $X_i \neq X_j, 1 \leq i < j \leq n$
    - $X_i + i \neq X_j + j, 1 \leq i < j \leq n$
    - $X_i - i \neq X_j - j, 1 \leq i < j \leq n$



- **Balanced Course Assignment Problem** At the beginning of the semester, the head of a computer science department *D* have to assign courses to teachers in a balanced way. The department *D* has *m* teachers $T=\{0, 2, ..., m-1\}$ and *n* courses $C=\{0, 2,..., n-1\}$. Each teacher $t \in T$ has a preference list which is a list of courses he/she can teach depending on his/her specialization. The preference information is represented by a 0-1 matrix $A_{m \times n}$ in which $A(t,c) = 1$ indicates that teacher t can teach the course c and $A(t,c) = 0$, otherwise. We know a set *B* of pairs of conflicting two courses that cannot be assigned to the same teacher as these courses have been already scheduled in the same slot of the timetable. The load of a teacher is the number of courses assigned to her/him. How to assign *n* courses to *m* teacher such that each course assigned to a teacher is in his/her preference list, no two conflicting courses are assigned to the same teacher, and the maximal load among teachers is minimal.
- **Modelling**
  - Decision variables
    - $X(i)$: teacher assigned to course i, $\forall i \in C$, domain $D(X(i)) = \{t \in T \mid A(t,i) = 1\}$
    - $Y(i)$: load of teacher i, domain $D(Y(i)) = \{0,1,...,n-1\}$
    - Z: maximum load among teachers
  - Constraints
    - $X(i) \neq X(j), \forall (i,j) \in B$
    - $Y(i) = \sum_{j \in C} (X(j) = i), \forall i \in T$
    - $Z \geq Y(i), \forall i \in T$
  - Objective function to be minimized: Z

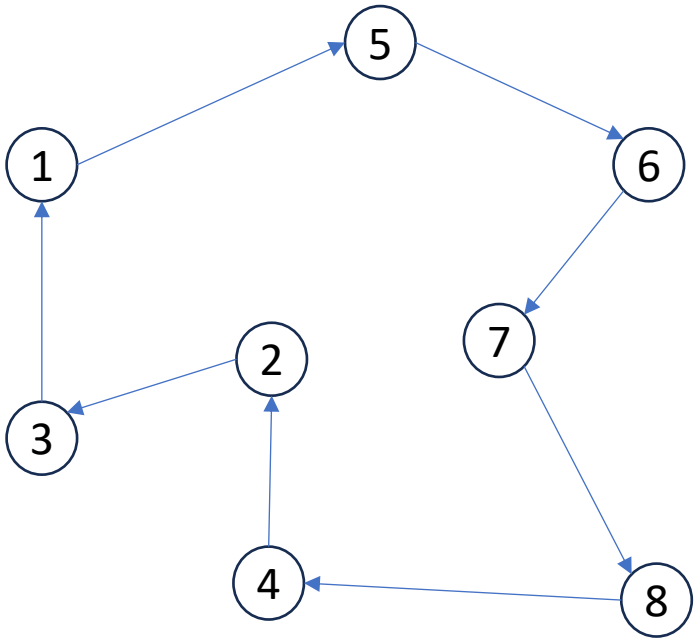
Introduction to combinatorial optimization & applications

- Applications
  - Manufacturing
  - Transportation & Logistics
  - Public services
  - Network Designs

The TSP problem

- **Problem statement** Given n points 1, 2, . . ., n. Denote C(i, j) the travel distance from point i to point j (i, j = 1, 2, . . ., n). Find the shortest closed tour starting from point 1, visiting each node 2, 3, . . ., n exactly once, and coming back to point 1.

	1	2	3	4	5	6	7	8
1	0	7	4	9	1	5	8	7
2	6	0	2	8	9	3	3	4
3	3	9	0	5	4	7	8	8
4	4	1	7	0	5	4	6	9
5	4	5	6	9	0	2	7	8
6	5	4	8	6	8	0	1	8
7	6	7	3	8	8	9	0	1
8	9	5	8	2	7	6	4	0



The TSP problem

- **Problem statement** Given n points 1, 2, . . ., n. Denote C(i, j) the travel distance from point i to point j (i, j = 1, 2, . . ., n). Find the shortest closed tour starting from point 1, visiting each node 2, 3, . . ., n exactly once, and coming back to point 1.

- Modelling: add an artificial point 0 representing the physical point 1, a solution is a route starting from 1, visiting 2, 3, ..., n exactly once, and terminating at 0

  - Decision variables
    - $X(i)$: subsequent point of the point i in the solution, $\forall\, i \in \{1, 2, ..., n\}$.
    - $Y(i)$: accumulate distance from point 1 to point i on the route, $\forall\, i \in \{1, 2, ..., n\}$
  - Domains: $X(i) \in \{1, 2, . . ., n, 0\}$, $Y(i) \in \{0, 1, . . ., M^*n\}$ with the constant $M = \max_{i,\, j \in \{1, 2, . . ., n\}} \{C(i, j)\}$
  - Constraints
    - $X(i) \neq X(j)$
    - $X(i) \neq i$
    - $Y(1) = 0$
    - $X(i) = j \Rightarrow Y(j) = Y(i) + C(i, j)$, $\forall\, i \in \{1, 2, ..., n\}$, $\forall\, j \in \{0, 1, 2, ..., n\}$
  - Objective function to be minimized

$$f(X, Y) = Y(0)$$

The TSP problem

- **Problem statement** Given n points 1, 2, . . ., n. Denote C(i, j) the travel distance from point i to point j (i, j = 1, 2, . . ., n). Find the shortest closed tour starting from point 1, visiting each node 2, 3, . . ., n exactly once, and coming back to point 1.

- Integer Linear Programming (ILP) model

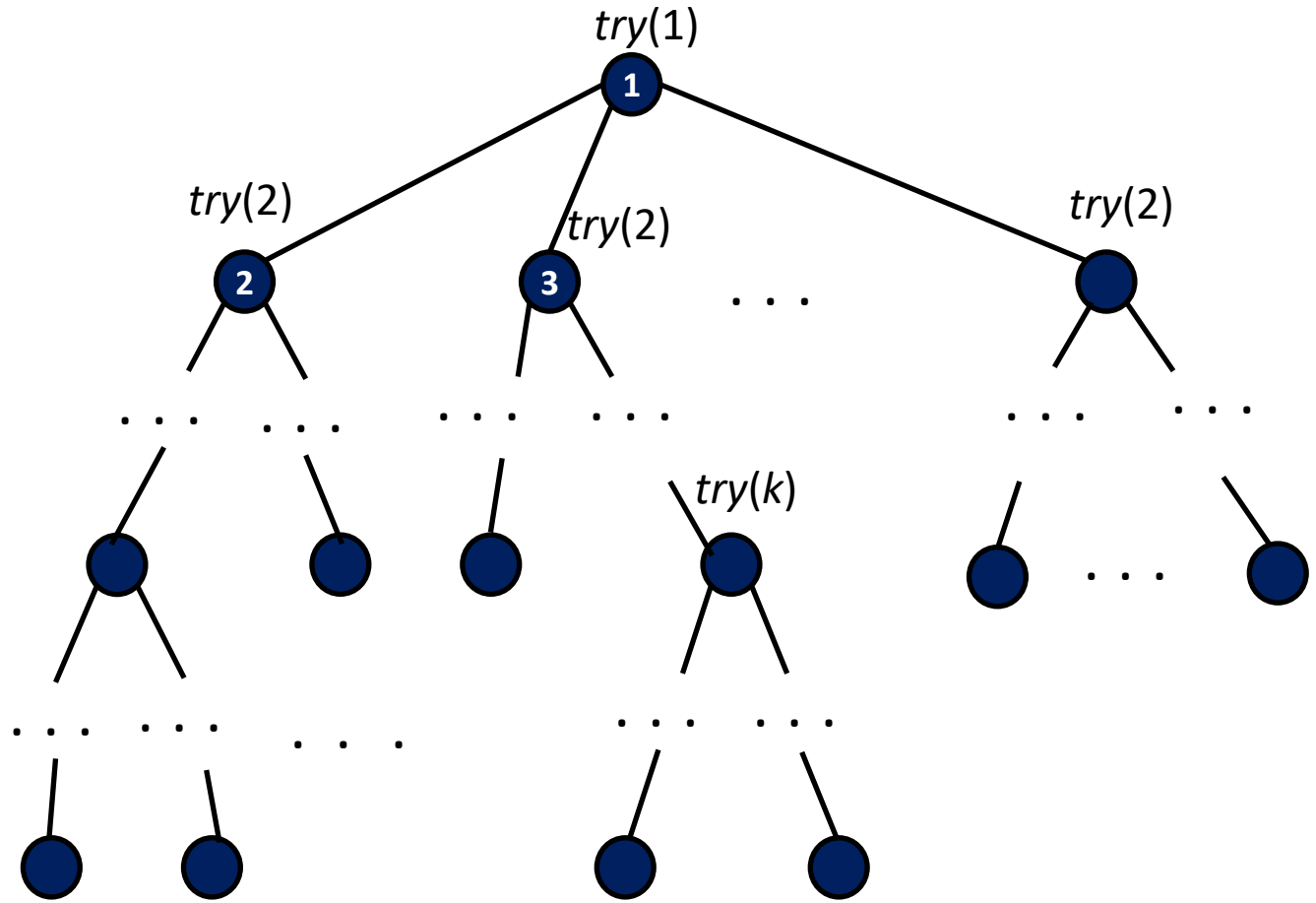
  - Decision variables
    - Binary variable $X(i,j)$ = 1 if the route traverses from point i to point j, and $X(i, j)$ = 0, otherwise.
  - Constraints
    - $\sum_{j=1}^{n} X(i,j) = \sum_{j=1}^{n} X(j,i) = 1, \forall\, i \in \{1,2,\ldots, n\}$
    - $\sum_{(i,j)\in S} X(i,j) \leq |S| - 1, \forall\, S \subseteq \{1, 2, \ldots, n\}$ and $|S| < n$
  - Objective function to be minimized

$$f(X) = \sum_{j=1}^{n} \sum_{i=1}^{n} C(i,j)X(i,j)$$

The TSP problem – exhaustive search

- Enumerate all possible solutions and keep track of the best one
- Solution representation: a permutation $X = (X_1, X_2, \ldots, X_n)$ in which X_i is the ith point in the route
  - Closed tour : $X_1 \rightarrow X_2 \rightarrow \ldots \rightarrow X_n \rightarrow X_1$
  - Total travel distance is $C(X_1, X_2) + C(X_2, X_3) + \ldots + C(X_n, X_1)$
- Exhaustive search
  - Scan all component $X_1, X_2, \ldots, X_n$. For each X_i
    - Explore all possible values for X_i
    - Use marking array to avoid the repetition of values in the solution:
      - visited[v]= True mean value v has appeared, and visited[v]= False, otherwise

The TSP problem – exhaustive search




```
try(k){ // try all values for X_k
  for v in A_k do {
     if check(v,k){
        X_k = v;
        visited[v] = True;
        if k = n then solution();
        else {
            try(k+1);
        }
        visited[v] = False;
     }
  }
}
```

The TSP problem – exhaustive search

```
import sys
[n] = [int(x) for x in sys.stdin.readline().split()]
C = []
for i in range(n):
 row = [int(x) for x in sys.stdin.readline().split()]
 C.append(row)
X = [0 for i in range(n)]
visited = [False for i in range(n)]
f = 0
f_min = 1e9
```

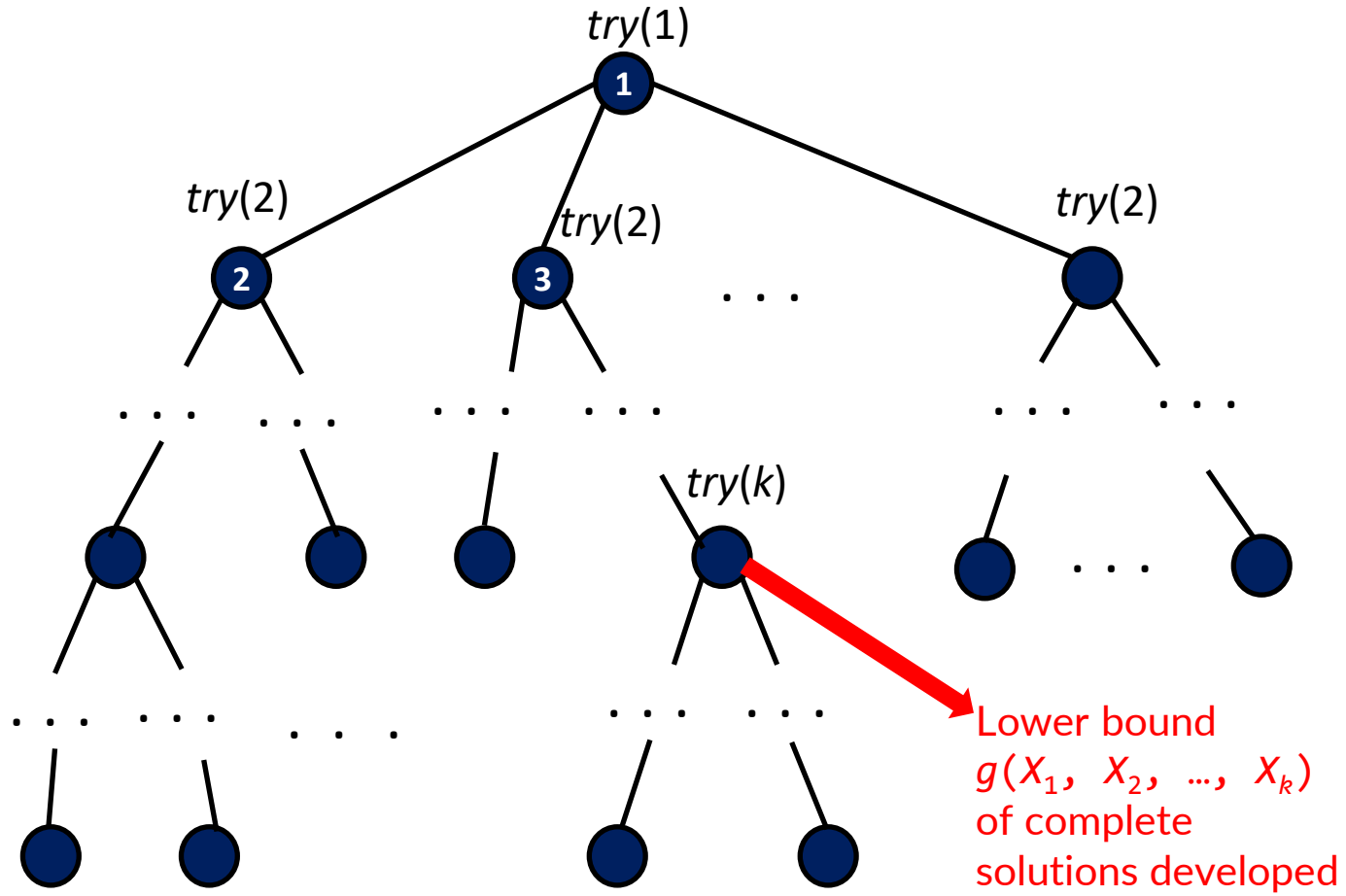
```
def solution():
 global f_min
 if f + C[X[n-1]][X[0]] < f_min:
  f_min = f + C[X[n-1]][X[0]]
  print('update best ',f_min)
```

The TSP problem – exhaustive search

```
def try_value(k):
 global f
 for v in range(n):
  if visited[v] == False:
   X[k] = v
   visited[v] = True
   f = f + C[X[k-1]][X[k]]
   if k == n-1:
    solution()
   else:
    try_value(k+1)
   visited[v] = False
   f = f - C[X[k-1]][X[k]]
```

```
f = 0
X[0] = 0
visited[0] = True
try_value(1)
```

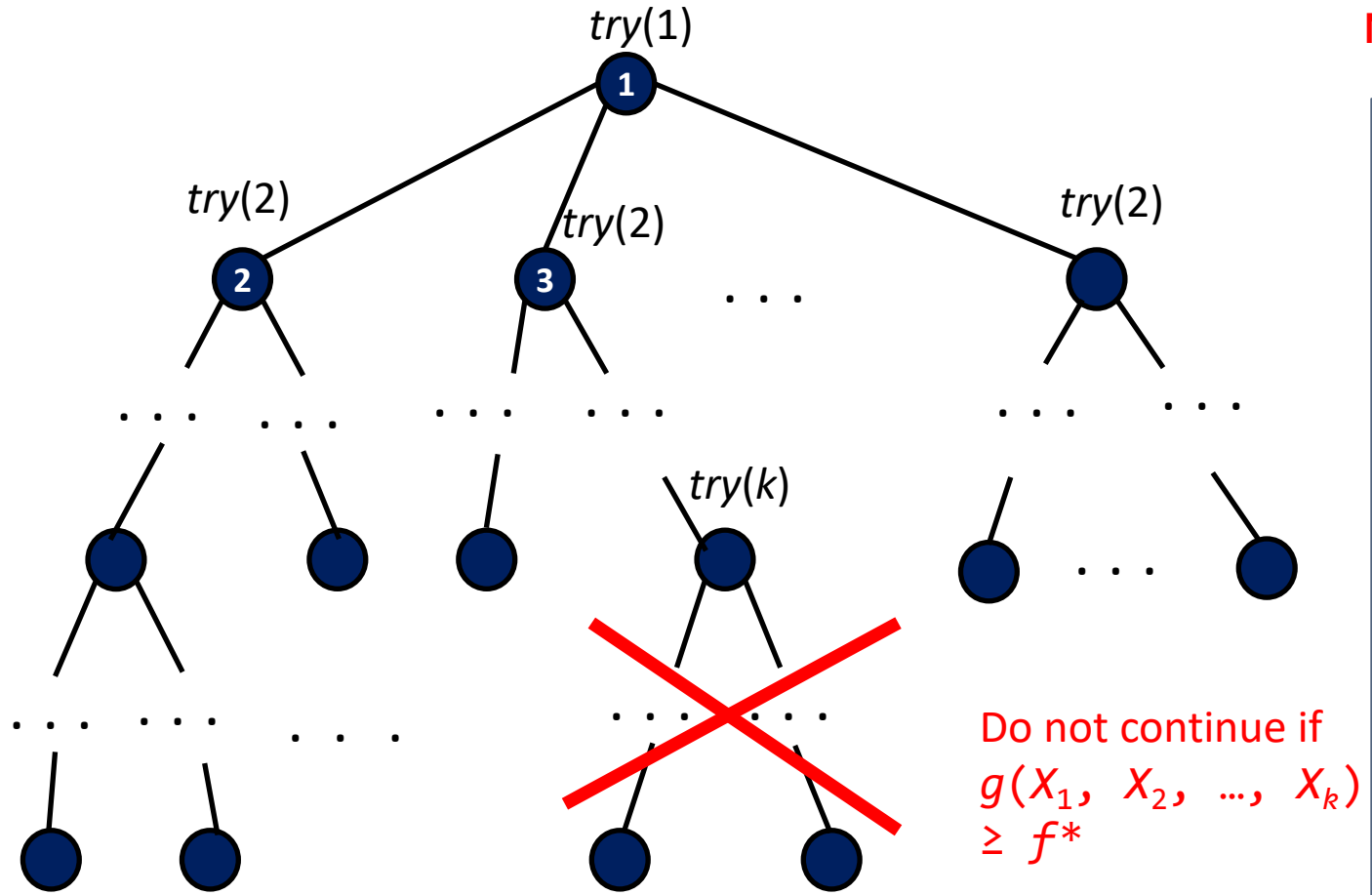
The TSP problem – branch and bound



Minimization problem (Let f^* be incumbent)

```
try(k){ // thử các giá trị có thể gán cho X_k
  for v in A_k do {
     if check(v,k){
        X_k = v;
        [Update a data structure D]
        if k = n then updateBest();
        else {
            if g(X_1, X_2, …, X_k) < f* then
                try(k+1);
        }
        [Recover the data structure D]
     }
  }
}
```

The TSP problem – branch and bound



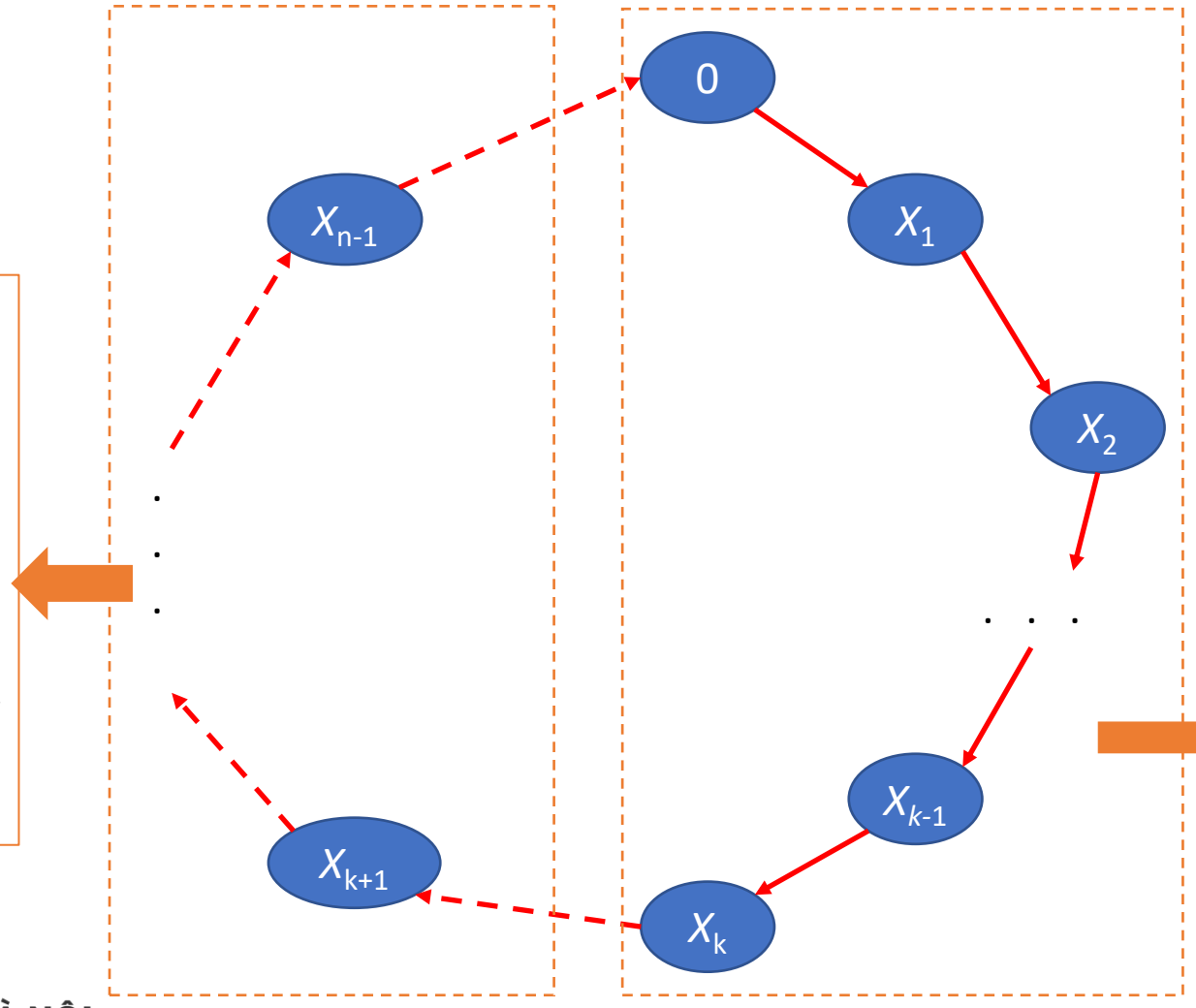
Do not continue if $g(X_1, X_2, \ldots, X_k) \geq f^*$

Minimization problem (Let f^* be incumbent)

```
try(k){ // thử các giá trị có thể gán cho X_k
  for v in A_k do {
     if check(v,k){
        X_k = v;
        [Update a data structure D]
        if k = n then updateBest();
        else {
           if g(X_1, X_2, …, X_k) < f* then
              try(k+1);
        }
        [Recover the data structure D]
     }
  }
}
```

The TSP problem – branch and bound

- Use lower bound evaluation for cutting off redundant computation



- Non-visited route consists of *n-k* segments, length of each segment is at least *Cmin*
- Length of the complete solution developed from the partial solution (X_0, X_1, . . ., X_k) is greater or equal to a lower bound $g = f + Cmin^*(n-k)$

Partial solution
- *k* segments
- Partial length *f*

The TSP problem – branch and bound

```
import sys
def computeMinC():
 minC = 1e9
 for i in range(n):
  for j in range(n):
   if i != j and minC > C[i][j]:
    minC = C[i][j]
 return minC
def solution():
 global f_min
 if f + C[X[n-1]][X[0]] < f_min:
  f_min = f + C[X[n-1]][X[0]]
  print('update best ',f_min)
```

```
def try_value(k):
 global f
 for v in range(n):
  if visited[v] == False:
   X[k] = v
   visited[v] = True
   f = f + C[X[k-1]][X[k]]
   if k == n-1:
    solution()
   else:
    if f + minC*(n-k) < f_min:
     try_value(k+1)
   visited[v] = False
   f = f - C[X[k-1]][X[k]]
```

The TSP problem – branch and bound

```
[n] = [int(x) for x in sys.stdin.readline().split()]

C = []

for i in range(n):

 row = [int(x) for x in sys.stdin.readline().split()]

 C.append(row)


minC = computeMinC()

X = [0 for i in range(n)]

visited = [False for i in range(n)]
```

```
f = 0

f_min = 1e9

X[0] = 0

visited[0] = True

try_value(1)

print('Shortest distance = ',f_min)
```

The TSP problem – cp model in or-tools

```
import sys

from ortools.sat.python import cp_model

[n] = [int(x) for x in sys.stdin.readline().split()]

C = []

row = [0 for i in range(n+1)]

C.append(row)

for i in range(n):

 row = [int(x) for x in sys.stdin.readline().split()]

 row.insert(0,0)

 C.append(row)

for i in range(1,n+1):

 C[0][i] = C[1][i]

 C[i][0] = C[i][1]
```

```
M = 0

for i in range(n+1):

 for j in range(n+1):

  if C[i][j] > M:

   M = C[i][j]


model = cp_model.CpModel()

X = {}

Y = {}

for i in range(1,n+1):

 X[i] = model.NewIntVar(0,n,'X(' +str(i) + ')')

for i in range(n+1):

 Y[i] = model.NewIntVar(0,M*n,'Y(' + str(i) + ')')
```

The TSP problem – cp model in or-tools

```
for i in range(1,n+1):
 model.Add(X[i] != i)
for i in range(1,n+1):
 for j in range(i+1,n+1):
  model.Add(X[i] != X[j])
model.Add(Y[1] == 0)
for i in range(1,n+1):
 for j in range(n+1):
  # state constraint: X[i] = j => Y[j] = Y[i] + C[i][j]
  b = model.NewBoolVar('')
  model.Add(X[i] == j).OnlyEnforceIf(b)
  model.Add(X[i] != j).OnlyEnforceIf(b.Not())
  model.Add(Y[j] == Y[i] + C[i][j]).OnlyEnforceIf(b)
model.Minimize(Y[0])
```

```
solver = cp_model.CpSolver()
#solver.parameters.max_time_in_seconds = 5.0
status = solver.Solve(model)

if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
 print(solver.Value(Y[0]))
 for i in range(1,n+1):
  print('X(' + str(i) + ') = ',solver.Value(X[i]))
 for i in range(n+1):
  print('Y(' + str(i) + ') = ',solver.Value(Y[i]))
else:
 print('NOT_FEASIBLE')
```

The TSP problem – ILP model in or-tools

- Problem statement Given n points 1, 2, . . ., n. Denote C(i, j) the travel distance from point i to point j (i, j = 1, 2, . . ., n). Find the shortest closed tour starting from point 1, visiting each node 2, 3, . . ., n exactly once, and coming back to point 1.

- Integer Linear Programming (ILP) model

  - Decision variables
    - Binary variable $X(i,j)$ = 1 if the route traverses from point i to point j, and $X(i, j)$ = 0, otherwise.
  - Constraints
    - $\sum_{j=1}^{n} X(i,j) = \sum_{j=1}^{n} X(j,i) = 1, \forall\, i \in \{1,2,\ldots, n\}$
    - $\sum_{(i,j)\in S} X(i,j) \leq |S| - 1, \forall\, S \subseteq \{1, 2, \ldots, n\}$ and $|S| < n$ (Sub-Tour Elimination Constraint - SEC)
  - Objective function to be minimized

$$f(X) = \sum_{j=1}^{n} \sum_{i=1}^{n} C(i,j)X(i,j)$$

The TSP problem – ILP model in or-tools

```
class SubSetGenerator:
 def __init__(self,N):
  self.N = N
  self.x = [0 for i in range(N+1)]
 def __CollectSubSet__(self):
  S = []
  for i in range(1,self.N+1):
   if self.x[i] == 1:
    S.append(i)
  return S


 def GenerateFirstSubSet(self):
  return self.__CollectSubSet__()
```

```
 def GenerateNextSubSet(self):
  N = self.N
  x = self.x
  i = N
  while i >= 1 and x[i] == 1:
   i = i - 1
  if i == 0:
   return None
  x[i] = 1
  for j in range(i+1,N+1):
   x[j] = 0
  return self.__CollectSubSet__()
```

The TSP problem – ILP model in or-tools

```
import sys
from ortools.linear_solver import pywraplp


def read_input():
 [N] = [int(x) for x in sys.stdin.readline().split()]
 d = []
 d.append([])
 for i in range(N):
  r = [int(x) for x in sys.stdin.readline().split()]
  r.insert(0,0)
  d.append(r)
 return N, d


N, d = read_input()
```

```
solver = pywraplp.Solver.CreateSolver('CBC')
X = [[solver.IntVar(0,1,'') for j in range(N+1)] for i in range(N+1)]


# flow balance constraint
for i in range(1, N+1):
 c = solver.Constraint(1,1)
 for j in range(1,N+1):
  if i != j:
   c.SetCoefficient(X[j][i],1)


 c = solver.Constraint(1,1)
 for j in range(1,N+1):
  if i != j:
   c.SetCoefficient(X[i][j],1)
```

The TSP problem – ILP model in or-tools

```
# state SEC (Sub-Tour Elimination Constraint)

SG = SubSetGenerator(N)

S = SG.GenerateFirstSubSet()

while True:

 if len(S) >= 2 and len(S) < N:

  c = solver.Constraint(0, len(S) - 1)

  for i in S:

   for j in S:

    if i != j:

     c.SetCoefficient(X[i][j],1)

 S = SG.GenerateNextSubSet()

 if S == None:

  break
```

```
# objective function

objective = solver.Objective()

for i in range(1,N+1):

 for j in range(1,N+1):

  if i != j:

   objective.SetCoefficient(X[i][j],d[i][j])


objective.SetMinimization()

result_status = solver.Solve()
```

The TSP problem – ILP model in or-tools

```
if result_status != pywraplp.Solver.OPTIMAL:
 print('cannot find optimal solution')
else:
 print('optimal objective value = ',solver.Objective().Value())

 for i in range(1,N+1):
  for j in range(1,N+1):
   if X[i][j].solution_value() > 0:
    print('(',i,',',j,')')
 route = extractSolutionRoute()
 print(route)
```

```
def findNext(current):
 for i in range(1,N+1):
  if X[current][i].solution_value() > 0:
   return i
 return -1 # not found


def extractSolutionRoute():
 route = [1]
 current = 1
 for i in range(2,N+1):
  nextPoint = findNext(current)
  route.append(nextPoint)
  current = nextPoint
 return route
```

The TSP problem – ILP model in or-tools with dynamic SEC

- Integer Linear Programming (ILP) model
  - Decision variables
    - Binary variable $X(i,j)$ = 1 if the route traverses from point i to point j, and $X(i, j)$ = 0, otherwise.
  - Constraints
    - $\sum_{j=1}^{n} X(i,j) = \sum_{j=1}^{n} X(j,i) = 1, \forall\, i \in \{1,2,\ldots, n\}$
    - $\sum_{(i,j)\in S} X(i,j) \leq |S| - 1, \forall\, S \subseteq \{1, 2, \ldots, n\}$ and $|S| < n$ (Sub-Tour Elimination Constraint - SEC)
  - Objective function to be minimized

$$f(X) = \sum_{j=1}^{n} \sum_{i=1}^{n} C(i,j)X(i,j)$$

- Dynamic SEC
  - **Do not** "include all SECs in the model, then solve the model"
  - Relax SECs: model without SECs
  - Repeat: detect sub-tours appearing in the solution, then add corresponding SECs to the model, then re-solve

The TSP problem – ILP model in or-tools with dynamic SEC

```
from ortools.linear_solver import pywraplp
import numpy as np
import random as rd
import sys

def create_solver_and_variables():
 solver = pywraplp.Solver.CreateSolver('SCIP')
 x = [[solver.IntVar(0,1,'') for i in range(n)] for j in range(n)]
 x = np.array(x)
 return solver,x

def create_balance_flow_constraint(solver,x):
 for i in range(n):
  cstr = solver.Constraint(1,1)
  for j in range(n):
   if i != j:
    cstr.SetCoefficient(x[i,j],1)
  cstr = solver.Constraint(1,1)
  for j in range(n):
   if i != j:
    cstr.SetCoefficient(x[j,i],1)
```

The TSP problem – ILP model in or-tools with dynamic SEC

```
def find_next(s,x):
 for i in range(n):
  if i != s and x[s,i].solution_value() > 0:
   return i;
 return -1#not found


def extract_cycle(x):
 starts = [i for i in range(n)]
 cycles = []
 while len(starts) > 0:
  s = starts[0]
  C = [s]
  starts.remove(s)
  while True:
   i = find_next(s,x)
   if i == -1:
    break
   if i in C:
    break
   C.append(i)
   starts.remove(i)
   s = i
  cycles.append(C)
 return cycles
```

The TSP problem – ILP model in or-tools with dynamic SEC

```
def solve_dynamic_SEC(SECs):
 solver,x = create_solver_and_variables()
 create_balance_flow_constraint(solver,x)

 #create sub-tour-elimination constraints in the list SECs
 for C in SECs:
  cstr = solver.Constraint(0,len(C) - 1)
  for i in C:
   for j in C:
    if i != j:
     cstr.SetCoefficient(x[i,j],1)
 obj = solver.Objective()
 for i in range(n):
  for j in range(n):
   if i != j:
    obj.SetCoefficient(x[i,j],d[i,j])

 rs = solver.Solve()

 cycles = extract_cycle(x)
 return cycles, solver.Objective().Value()
```

The TSP problem – ILP model in or-tools with dynamic SEC

```
def solve_TSP():
 SECs = []
 while True:
  cycles,opt = solve_dynamic_SEC(SECs)
  print('found obj = ',opt,'number of sub-tours = ',len(cycles))
  if len(cycles) == 1: #found optimal
   print('found optimal!!!')
   return cycles[0]
  for C in cycles:
   SECs.append(C)
   print('add SEC ',C)
 return None

[n] = [int(x) for x in sys.stdin.readline().split()]
d = []
for i in range(n):
 r = [int(x) for x in sys.stdin.readline().split()]
 d.append(r)
d = np.array(d,dtype='double')

res = solve_TSP()
```

The TSP problem – experiments

Instances	Exhaustive search	Branch & bound	Cp model	ILP model	ILP with Dynamic SEC
tsp-5	0.00026	0.00034	0.74	1.35	0.023
tsp-10	0.77	0.012	17.95	1.5	0.043
tsp-15	-	0.57	-	18.17	0.10
tsp-20	-	29.85	-	-	0.20
tsp-30	-	-	-	-	0.20
tsp-50	-	-	-	-	0.25
tp-100	-	-	-	-	4.58

The TSP problem – incomplete methods

- Exact (complete) methods cannot handle large-scale instances
- Incomplete methods aim at finding high-quality solutions in a reasonable time
  - Heuristics
    - Greedy
    - Hill Climbing
  - Metaheuristics
    - Tabu Search
    - Simulated Annealing
    - Ant Colony Optimization
    - Genetic Algorithms
    - etc.
  - Learning-based methods: Reinforcement Learning

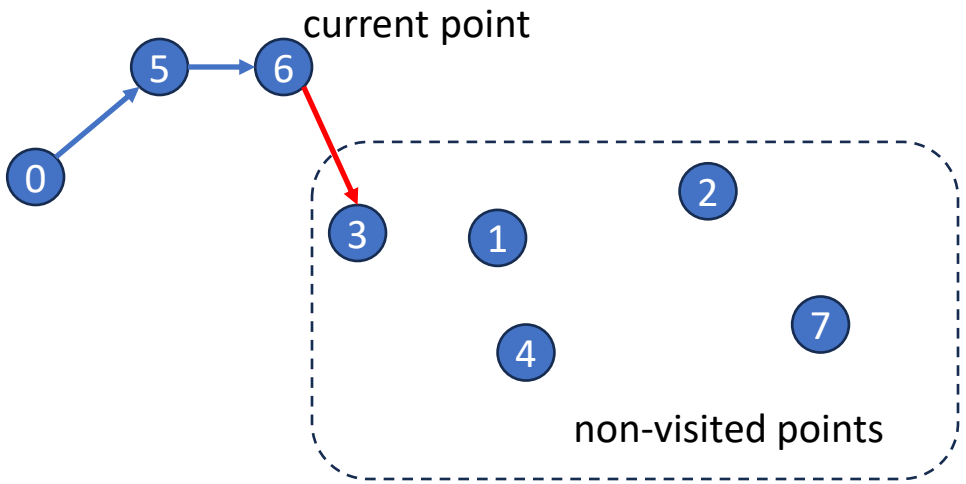
The TSP problem – greedy algorithms

- Problem-solving strategy that makes the optimal choice at each immediate step with the hope of finding the overall best solution.
- Greedy algorithms focus entirely on **local optimization** without worrying about future consequences or past choices
- Notations:
  - $\boldsymbol{S}$: Solution to find
  - $\boldsymbol{C}$: A set of candidates
  - $\boldsymbol{select(C)}$: A function to select the best candidate
  - $\boldsymbol{solution(S)}$: A function returns TRUE if S is a validate solution, otherwise returns FALSE
  - $\boldsymbol{feasible(S)}$: A function returns TRUE if S does not violate constraints, otherwise return FALSE

```
Greedy() {
  S = ∅;
  while C ≠ ∅ and not solution(S){
    x = select(C);
    C = C \ {x};
    if feasible(S ∪ {x}) {
      S = S ∪ {x};
    }
  }
  return S;
}
```

The TSP problem – nearest neighbor greedy algorithm

- Each step: move to the nearest point to the current point



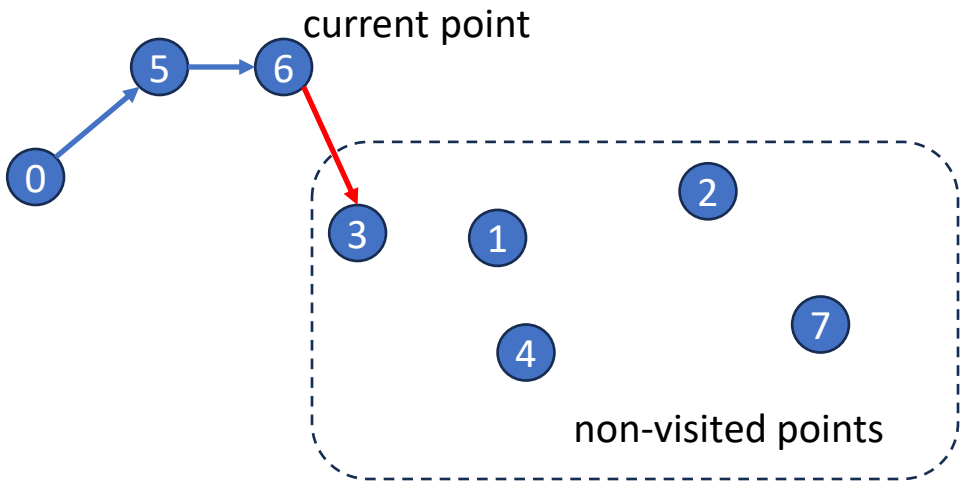

```
import sys
import random as rd
import time

def input_data_file(filename):
 with open(filename,'r') as f:
  [n] = [int(x) for x in f.readline().split()]
  C = []
  for i in range(n):
   r = [int(x) for x in f.readline().split()]
   C.append(r)
  return n, C

def select_nearest(cand, cur):
 minD = 1e9
 sel_i = -1
 for i in cand:
  if C[cur][i] < minD:
   minD = C[cur][i]
   sel_i = i
 return sel_i
```

The TSP problem – nearest neighbor greedy algorithm

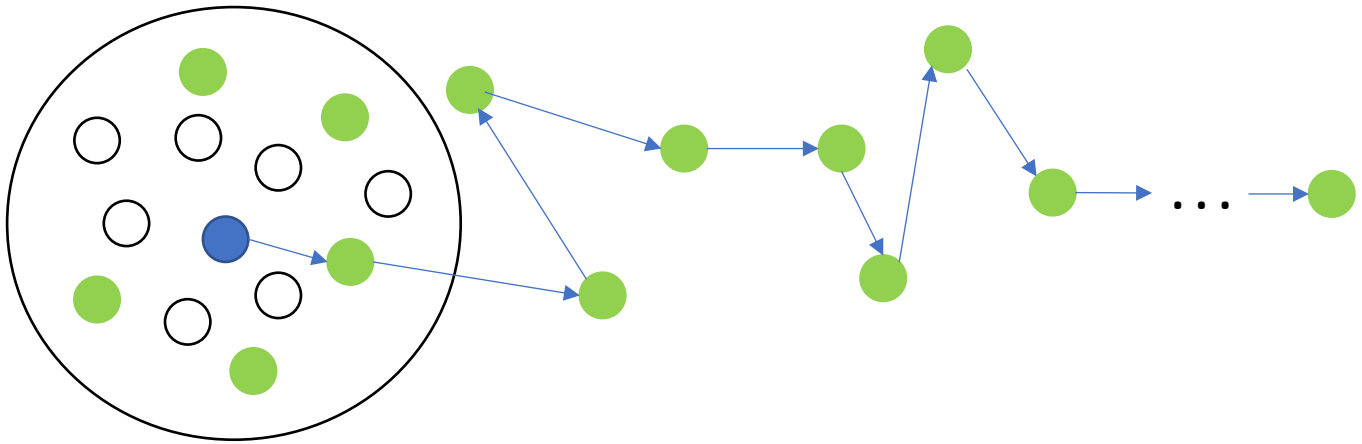
- Each step: move to the nearest point to the current point




```
def nearest_neighbor_greedy():
 global f
 x = [0]
 cand = set()
 for i in range(1,n):
  cand.add(i)
 cur = 0
 f = 0 # length of the tour
 while len(cand) > 0:
  sel_i = select_nearest(cand,cur)
  x.append(sel_i)
  f += C[cur][sel_i]
  cur = sel_i
  cand.remove(sel_i)
 f += C[cur][0]
 return x,f
```

The TSP problem – local search

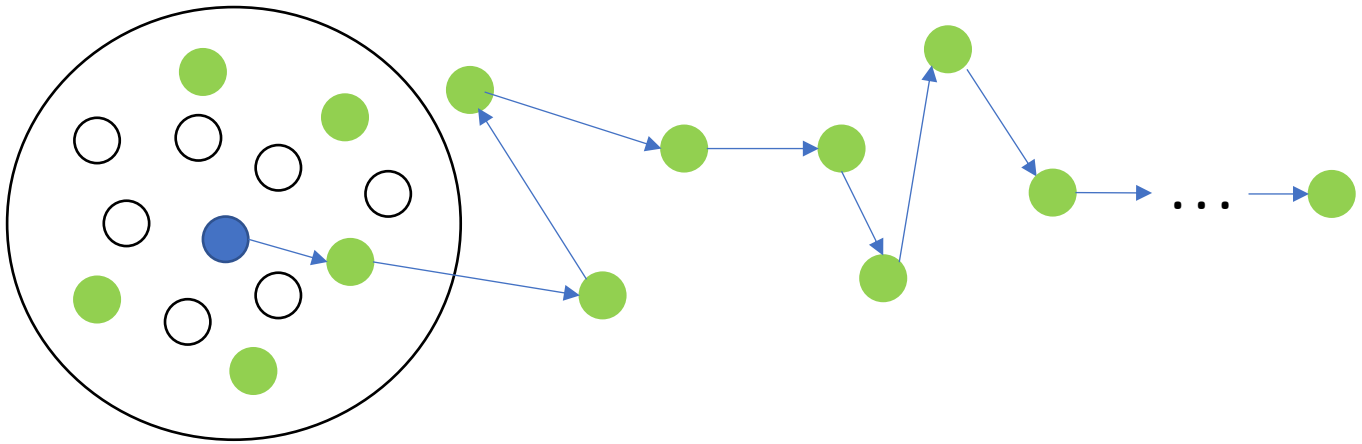
- Method for finding high-quality solutions to optimization problems
- Key idea
  - Generate an initial solution
  - Iteratively move from a current solution to one of its neighbors



```
function LocalSearch(N, f){
  // N: neighborhood
  // f: quality function
  s = Generate an initial solution;
  s* = s;
  while termination not reach do {
    s = Select(N(s), s); //neighbor selection
    if s = NULL then break;
    if s is better than s* then
      s* = s;
  }
  return s*;
}
```

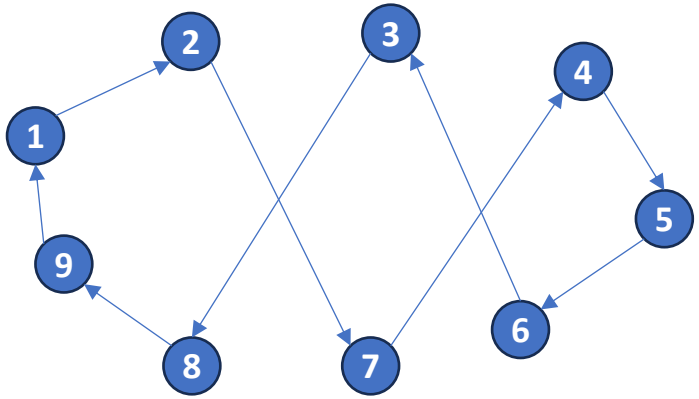
The TSP problem – local search

- Local move operators
  - Small change on the current solution to generate a neighboring solution
    - Update the value of a selected component (variable)
    - Swap the value of 2 selected variables
    - ...

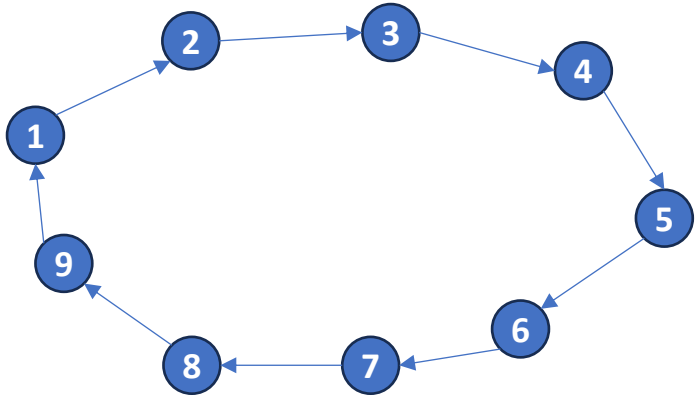


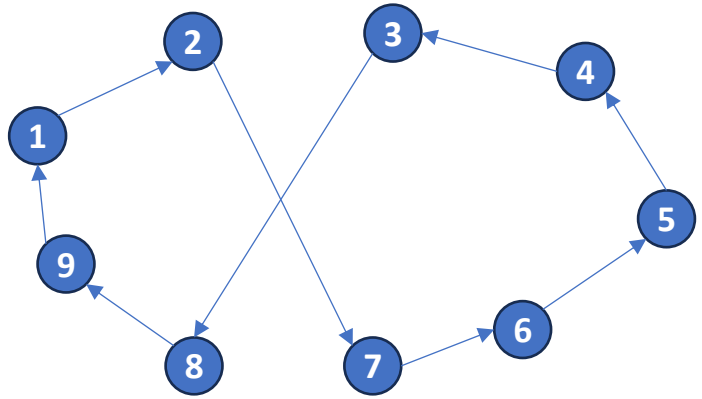
```
function LocalSearch(N, f){
  // N: neighborhood
  // f: quality function
  s = Generate an initial solution;
  s* = s;
  while termination not reach do {
    s = Select(N(s), s); //neighbor selection
    if s = NULL then break;
    if s is better than s* then
      s* = s;
  }
  return s*;
}
```

The TSP problem – local search

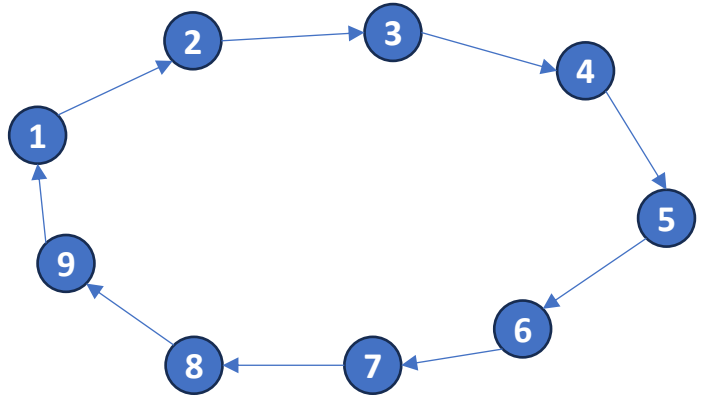


Swap (3, 7)





2-Opt (2, 3)



The TSP problem – local search

Feature	Hill Climbing	Tabu Search
Strategy	Greedy; always moves to the best immediate neighbor.	Uses a memory-based strategy; can move to non-improving neighbors.
Handling Local Optima	**Poor.** It gets stuck immediately when it reaches a peak where all neighbors are worse.	**Good.** It allows moves to worse solutions (uphill moves in minimization) to escape local traps.
Aspiration Criteria	Not applicable.	Used to override tabu restrictions if a forbidden move leads to a solution better than the best-known one so far.
Risk of Cycling	High (can get trapped oscillating between states if not randomized).	Low, because the tabu list explicitly prevents revisiting recent states.

The TSP problem – swap-based hill climbing

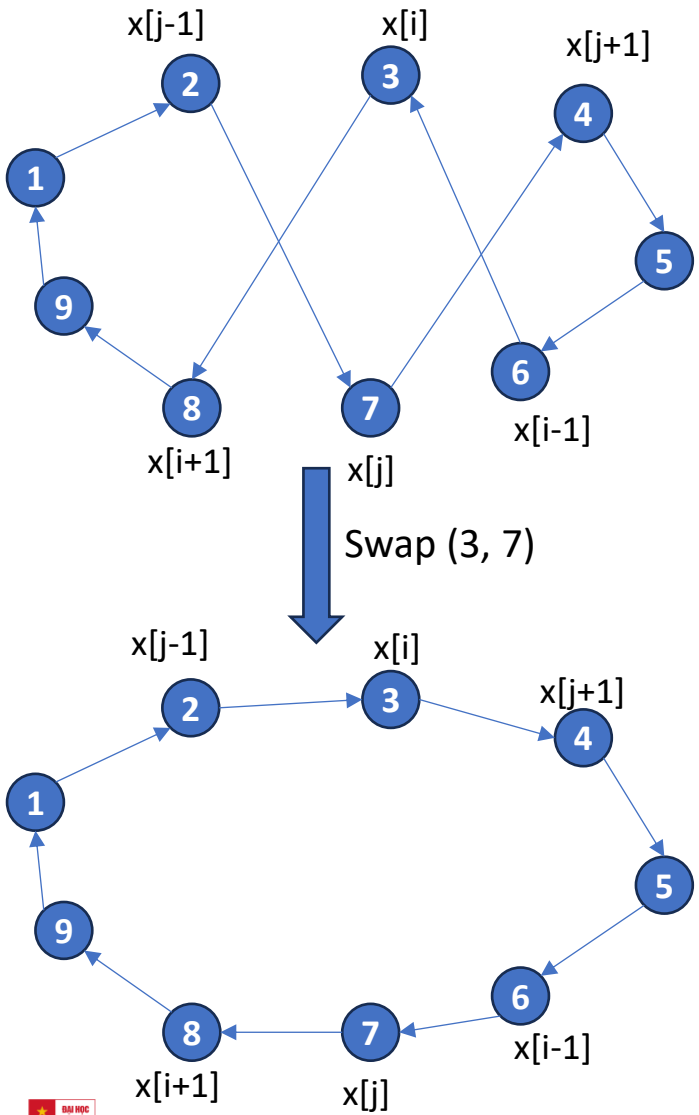
```
import sys
import random as rd
import time
import math
def input_data_file(filename):
 with open(filename,'r') as f:
  [n] = [int(x) for x in f.readline().split()]
  C = []
  for i in range(n):
   r = [int(x) for x in f.readline().split()]
   C.append(r)
  return n, C

def select_nearest(cand, cur):
 minD = 1e9
 sel_i = -1
 for i in cand:
  if C[cur][i] < minD:
   minD = C[cur][i]
   sel_i = i
 return sel_i
```

The TSP problem – swap-based hill climbing

```
def greedy_initial_solution():
 global f
 x = [0]
 cand = set()
 for i in range(1,n):
  cand.add(i)
 cur = 0
 L = 0
 while len(cand) > 0:
  sel_i =  select_nearest(cand, cur)
  x.append(sel_i)
  L += C[cur][sel_i]
  cur = sel_i
  cand.remove(sel_i)
 L += C[cur][0]
 return x,L
```

The TSP problem – swap-based hill climbing




```
def get_swap_delta(i,j):
 # return the differenciation of the length of the tour if points x[i] and x[j] are swapped
 delta = 0
 delta = delta - C[x[i-1]][x[i]] - C[x[i]][x[i+1]] - C[x[j-1]][x[j]] - C[x[j]][x[j+1]]
 delta = delta + C[x[i-1]][x[j]] + C[x[j]][x[i+1]] + C[x[j-1]][x[i]] + C[x[i]][x[j+1]]
 return delta

def perform_move(i,j):
 global f,x
 f = f - C[x[i-1]][x[i]] - C[x[i]][x[i+1]] - C[x[j-1]][x[j]] - C[x[j]][x[j+1]]
 f = f + C[x[i-1]][x[j]] + C[x[j]][x[i+1]] + C[x[j-1]][x[i]] + C[x[i]][x[j+1]]
 x[i],x[j] = x[j],x[i]
 return
```

The TSP problem – swap-based hill climbing

```
def hill_climbing_select(it):
 # explore all pairs (i,j) for swap
 cand = []
 min_delta = 1e9
 for i in range(1,n-1):
  for j in range(i+2,n-1):
   delta = get_swap_delta(i,j)
   if delta < 0:
    if delta < min_delta:
     min_delta = delta
     cand = [[i,j]]
    elif delta == min_delta:
     cand.append([i,j])
 if len(cand) == 0:# not found any move
  return -1,-1,0

 idx = rd.randint(0,len(cand)-1)
 [sel_i,sel_j] = cand[idx]

 return sel_i,sel_j,min_delta
```

The TSP problem – swap-based hill climbing

```
def hill_climbing(max_iterations, time_limit):
 global x, f, f_best
 start_time = time.time()
 x, f = greedy_initial_solution()
 f_best = f
 old_f = f
 for it in range(max_iterations):
  end_time = time.time()
  if end_time - start_time > time_limit:
   print('time expired -> break')
   break
  i,j,delta = hill_climbing_select(it)
  if i < 0: # no move is selected
   break
  perform_move(i,j)
  if delta >= 0:
   print('step ',it,' UP, f = ',f,' best = ',f_best)
  else:
   print('step ',it,' DOWN, f = ',f,' best = ',f_best)

  if f < f_best:
   f_best = f
   print('step ',it,': update best ',f_best)
   for i in range(n):
    x_best[i] = x[i]

 return f_best, x_best
```

The TSP problem – swap-based tabu search

```
import sys
import random as rd
import time
import math
def input_data_file(filename):
 with open(filename,'r') as f:
  [n] = [int(x) for x in f.readline().split()]
  C = []
  for i in range(n):
   r = [int(x) for x in f.readline().split()]
   C.append(r)
  return n, C

def select_nearest(cand, cur):
 minD = 1e9
 sel_i = -1
 for i in cand:
  if C[cur][i] < minD:
   minD = C[cur][i]
   sel_i = i
 return sel_i
```

The TSP problem – swap-based tabu search

```
def greedy_initial_solution():
 global f
 x = [0]
 cand = set()
 for i in range(1,n):
  cand.add(i)
 cur = 0
 L = 0
 while len(cand) > 0:
  sel_i =  select_nearest(cand, cur)
  x.append(sel_i)
  L += C[cur][sel_i]
  cur = sel_i
  cand.remove(sel_i)
 L += C[cur][0]
 return x,L
```

The TSP problem – swap-based tabu search

```
def get_swap_delta(i,j):
 # return the differenciation of the length of the tour if points x[i] and x[j] are swapped
 delta = 0
 delta = delta - C[x[i-1]][x[i]] - C[x[i]][x[i+1]] - C[x[j-1]][x[j]] - C[x[j]][x[j+1]]
 delta = delta + C[x[i-1]][x[j]] + C[x[j]][x[i+1]] + C[x[j-1]][x[i]] + C[x[i]][x[j+1]]
 return delta

def perform_move(i,j):
 global f,x
 f = f - C[x[i-1]][x[i]] - C[x[i]][x[i+1]] - C[x[j-1]][x[j]] - C[x[j]][x[j+1]]
 f = f + C[x[i-1]][x[j]] + C[x[j]][x[i+1]] + C[x[j-1]][x[i]] + C[x[i]][x[j+1]]
 x[i],x[j] = x[j],x[i]
 return
```

The TSP problem – swap-based tabu search

```
def tabu_select(it):
 # explore all pairs (i,j) for swap,
 # consider (i,j) not tabu or improve the best solution found so far
 cand = []
 min_delta = 1e9
 for i in range(1,n-1):
  for j in range(i+2,n-1):
   delta = get_swap_delta(i,j)
   if tabu[i][j] <= it or f + delta < f_best:
    if delta < min_delta:
     min_delta = delta
     cand = [[i,j]]
    elif delta == min_delta:
     cand.append([i,j])
 if len(cand) == 0:# not found any move
  return -1,-1,0

 idx = rd.randint(0,len(cand)-1)
 [sel_i,sel_j] = cand[idx]

 return sel_i,sel_j,min_delta
```

The TSP problem – swap-based tabu search

```
def tabu_search(max_iterations,time_limit):
 global x, f, f_best
 start_time = time.time()
 x, f = greedy_initial_solution()

 for it in range(max_iterations):
  end_time = time.time()
  if end_time -start_time > time_limit:
   print('Time expired BREAK')
   break

  i,j,delta = tabu_select(it)
  if i < 0: # no move is selected
   break
  perform_move(i,j)
  if delta >= 0:
   print('step ',it,' UP, f = ',f,' best = ',f_best)
  else:
   print('step ',it,' DOWN, f = ',f,' best = ',f_best)

  if f < f_best:
   f_best = f
   print('step ',it,': update best ',f_best)
   for i in range(n):
    x_best[i] = x[i]
  # update tabu list: the move (i,j) is tabu from current iteration it until it + tbl
  tabu[i][j] = it + tbl

 return f_best, x_best
```

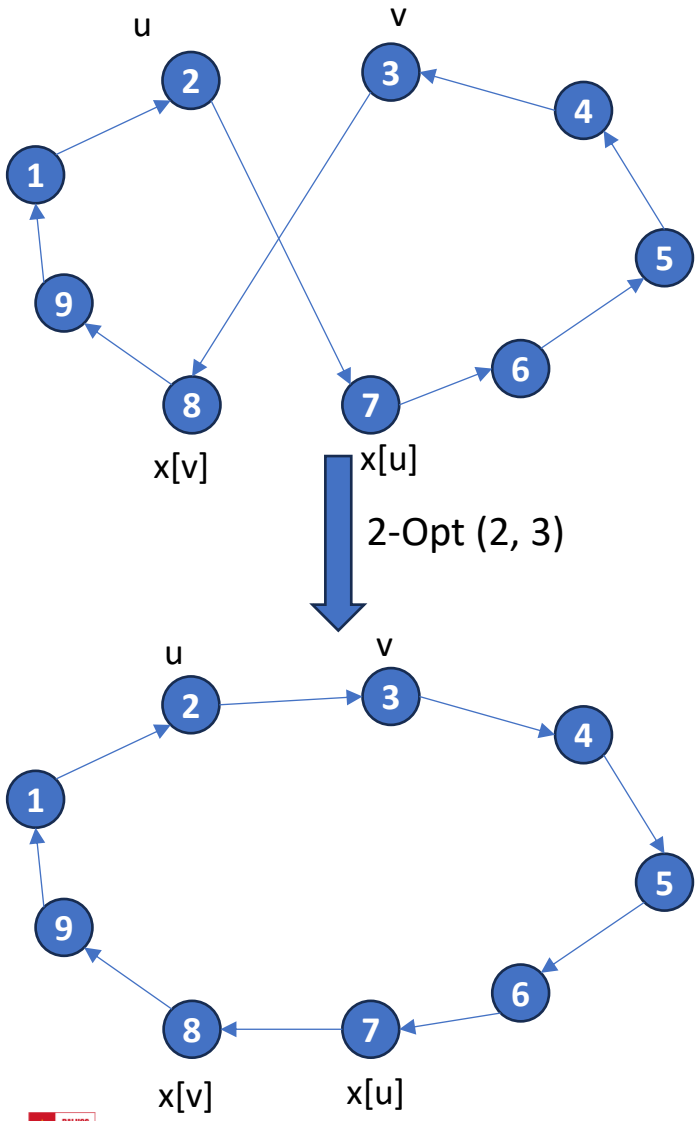
The TSP problem – two-opt hill climbing

```
import sys
import random as rd
import time
import math

def input_file(filename):
 with open(filename,'r') as f:
  [n] = [int(x) for x in f.readline().split()]
  C = []
  for i in range(n):
   r = [int(x) for x in f.readline().split()]
   C.append(r)
  return n,C

def init():
 global x, dl, dr, f_best, f
 x = [i+1 for i in range(n)]# x[i] is the next point of i
 x[n-1] = 0
 dr = [0 for i in range(n)]
 dl = [0 for i in range(n)]
 f = 0
 start = 0
 f_best = 1e9
```

The TSP problem – two-opt hill climbing




```
def get_two_opt_delta(u,v):
 nu = x[u]
 nv = x[v]
 duv = dr[v] - dr[nu]
 dvu = dl[v] - dl[nu]
 d1 = C[u][v] + C[nu][nv] - C[u][nu] - C[v][nv]
 d2 = dvu - duv
 return d1 + d2

def propagate():
 global f,dr,dl
 dr[0] = 0
 dl[0] = 0
 i = start
 f = 0
 while x[i] != start:
  dr[x[i]] = dr[i] + C[i][x[i]]
  dl[x[i]] = dl[i] + C[x[i]][i]
  f += C[i][x[i]]
  i = x[i]
 f += C[i][start]
```

The TSP problem – two-opt hill climbing

```
def two_opt_move(u, v):
 global x
 # pre-condition: u is locaited before v on the route
 # remove(u, x[u]) and (v, x[v])
 # reconnect (u, v) and (nu, nv)
 # reverse route (nu,...,v)
 nu = x[u]
 nv = x[v]
 p = nu
 pp = -1
 np = -1
 while p != nv:
  np = x[p]
  x[p] = pp
  pp = p
  p = np
 x[u] = v
 x[nu] = nv
 propagate()
```

The TSP problem – two-opt hill climbing

```
def select_nearest(cand, cur):
 minD = 1e9
 sel_i = -1
 for i in cand:
  if C[cur][i] < minD:
   minD = C[cur][i]
   sel_i = i
 return sel_i

def greedy_initial_solution():
 global f
 x = [0]
 cand = set()
 for i in range(1,n):
  cand.add(i)
 cur = 0
 L = 0
 while len(cand) > 0:
  sel_i =  select_nearest(cand, cur)
  x.append(sel_i)
  L += C[cur][sel_i]
  cur = sel_i
  cand.remove(sel_i)
 L += C[cur][0]
 return x,L
```

The TSP problem – two-opt hill climbing

```
def gen_initial_solution():
 global x, start
 minL = 1e9
 route = None
 route, minL = greedy_initial_solution()
 start = route[0]
 cur = start
 for i in route:
  if i != start:
   x[cur] = i
   cur = i
 x[cur] = start
 propagate()
```

The TSP problem – two-opt hill climbing

```
def hill_climbing_select(it):
 u = 0
 minDelta = 1e9
 sel_u = -1
 sel_v = -1
 cand = []
 while x[u] != start:
  nu = x[u]
  v = x[nu]
  while v != start:
   delta = get_two_opt_delta(u,v)
   if delta < minDelta and delta < 0:
    cand = [[u,v]]
    minDelta = delta
   elif delta == minDelta:
    cand.append([u,v])
   v = x[v]
  u = x[u]
 if len(cand) <= 0:
  return -1,-1,-1
 idx = rd.randint(0,len(cand)-1)
 [sel_u, sel_v] = cand[idx]
 return sel_u, sel_v, minDelta
```

The TSP problem – two-opt hill climbing

```
def solve(maxIter, time_limit):
 global f,f_best
 start_time = time.time()
 gen_initial_solution()
 f_best= f
 # local search
 for iter in range(maxIter):
  end_time = time.time()
  if end_time - start_time > time_limit:
   print('Time expired BREAK')
   break

  sel_u, sel_v, minDelta = hill_climbing_select(iter)
  if sel_u == -1:
   print('No move -> break')
   break
  two_opt_move(sel_u,sel_v)
  if f < f_best:
   f_best = f

  print(iter,',\t',f,',\t',f_best,',\t',' move(',sel_u,',',sel_v,') DOWN')
```

The TSP problem – two-opt tabu search

```
import sys
import random as rd
import time
import math

def input_file(filename):
 with open(filename,'r') as f:
  [n] = [int(x) for x in f.readline().split()]
  C = []
  for i in range(n):
   r = [int(x) for x in f.readline().split()]
   C.append(r)
  return n,C

def init():
 global x, dl, dr, f_best, f
 x = [i+1 for i in range(n)]# x[i] is the next point of i
 x[n-1] = 0
 dr = [0 for i in range(n)] # dr[i]: length of sub-route from the start to i on the solution route
 dl = [0 for i in range(n)] # dl[i]: length of sub-route from i back to the start (reverse route)
                            # on the solution route
 f = 0
 start = 0
 f_best = 1e9
```

The TSP problem – two-opt tabu search

```
def get_two_opt_delta(u,v):
 nu = x[u]
 nv = x[v]
 duv = dr[v] - dr[nu]
 dvu = dl[v] - dl[nu]
 d1 = C[u][v] + C[nu][nv] - C[u][nu] - C[v][nv]
 d2 = dvu - duv
 return d1 + d2

def propagate():
 global f,dr,dl
 dr[0] = 0
 dl[0] = 0
 i = start
 f = 0
 while x[i] != start:
  dr[x[i]] = dr[i] + C[i][x[i]]
  dl[x[i]] = dl[i] + C[x[i]][i]
  f += C[i][x[i]]
  i = x[i]
 f += C[i][start]
```

The TSP problem – two-opt tabu search

```
def two_opt_move(u, v):
 global x
 # pre-condition: u is locaited before v on the route
 # remove(u, x[u]) and (v, x[v])
 # reconnect (u, v) and (nu, nv)
 # reverse route (nu,...,v)
 nu = x[u]
 nv = x[v]
 p = nu
 pp = -1
 np = -1
 while p != nv:
  np = x[p]
  x[p] = pp
  pp = p
  p = np
 x[u] = v
 x[nu] = nv
 propagate()
```

The TSP problem – two-opt tabu search

```
def select_nearest(cand, cur):
 minD = 1e9
 sel_i = -1
 for i in cand:
  if C[cur][i] < minD:
   minD = C[cur][i]
   sel_i = i
 return sel_i

def greedy_initial_solution():
 global f
 x = [0]
 cand = set()
 for i in range(1,n):
  cand.add(i)
 cur = 0
 L = 0
 while len(cand) > 0:
  sel_i =  select_nearest(cand, cur)
  x.append(sel_i)
  L += C[cur][sel_i]
  cur = sel_i
  cand.remove(sel_i)
 L += C[cur][0]
 return x,L
```

The TSP problem – two-opt tabu search

```
def gen_initial_solution():
 global x, start
 minL = 1e9
 route = None
 route, minL = greedy_initial_solution()
 start = route[0]
 cur = start
 for i in route:
  if i != start:
   x[cur] = i
   cur = i
 x[cur] = start
 propagate()
```

The TSP problem – two-opt tabu search

```
def tabu_select(it):
 u = 0
 minDelta = 1e9
 sel_u = -1
 sel_v = -1
 cand = []
 while x[u] != start:
  nu = x[u]
  v = x[nu]
  while v != start:
   delta = get_two_opt_delta(u,v)
   if tabu[u][v] <= it or f + delta < f_best:
    if delta < minDelta:
     cand = [[u,v]]
     minDelta = delta
    elif delta == minDelta:
     cand.append([u,v])
   v = x[v]
  u = x[u]
 if len(cand) <= 0:
  return -1,-1,-1
 idx = rd.randint(0,len(cand)-1)
 [sel_u, sel_v] = cand[idx]
 return sel_u, sel_v, minDelta
```

The TSP problem – two-opt tabu search

```
def solve(maxIter, time_limit):
 global f,f_best
 start_time = time.time()
 gen_initial_solution()
 f_best= f
 # local search
 for iter in range(maxIter):
  end_time = time.time()
  if end_time - start_time > time_limit:
   print('Time expired BREAK')
   break

  sel_u, sel_v, minDelta = tabu_select(iter)
  if sel_u == -1:
   print('No move -> break')
   break
  two_opt_move(sel_u,sel_v)
  tabu[sel_u][sel_v] = iter + tbl

  if f < f_best:
   f_best = f

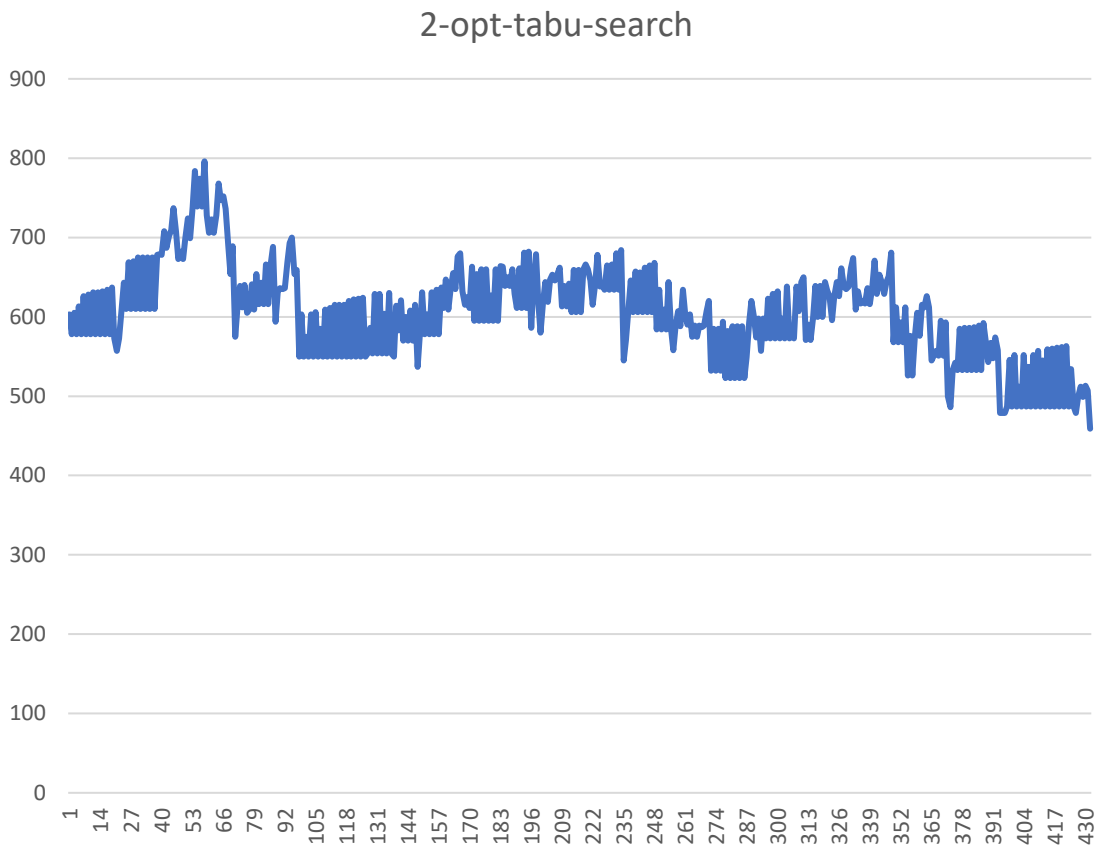
  if minDelta >= 0:
   print(iter,',\t',f,',\t',f_best,',\t',' move(',sel_u,',',sel_v,') UP')
  else:
   print(iter,',\t',f,',\t',f_best,',\t',' move(',sel_u,',',sel_v,') DOWN')
```

The TSP problem – experiments

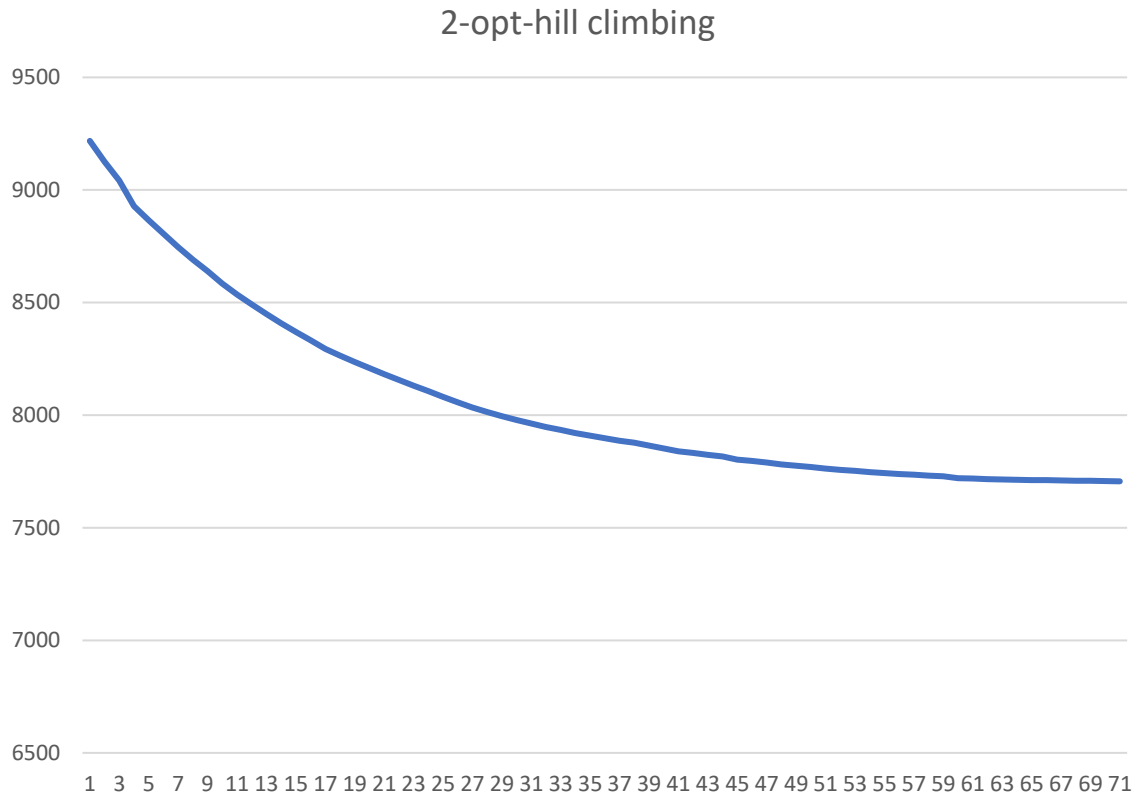
data	greedy-nearest neighbor	swap-base-hill-climbing				swap-base-tabu-search				2-opt-hill-cimbing				2-opt-tabusearch			
	min	min	max	avg	std	min	max	avg	std	min	max	avg	std	min	max	avg	std
tsp-30	54	54	54	54	0	47	49	48.1	0.35	47	47	47	0	41	47	45.9	0.03
tsp-50	96592	85040	85040	85040	0	83338	83978	83914	20.24	79781	79781	79781	0	72473	72473	72473	0
tsp-100	578	537	537	537	0	516	537	532.1	1.23	578	578	578	0	482	557	516.9	1.29
tsp-400	9346	9167	9167	9167	0	9115	9133	9121.3	0.85	7706	7706	7706	0	7646	7696	7671	7.9
tsp-500	10119	9859	9859	9859	0	9855	9855	9855	0	8590	8627	8619.6	2.34	8538	8596	8554.3	5.15

10 runs, time limit each run is 60s

The TSP problem – experiments



2-opt Tabu search behavior on instance tsp-100



2-opt Hill climbing behavior on instance tsp-400

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !